IMT Atlantique

Projet Procom

Argus

Historique du développement

SCHOBERT Néo
EL AKRABA Othmane
AYOUBI Houcine
TRAORE Idrissa

Argus – Debug Story

Table des matières

I Genèse et motivations 2

II Construction du graphe d'appels et premières frictions 2

III Du code complet au BackSlicer 3

IV Adaptation aux modèles on-premise 3

V Le workflow de benchmarking 4

VI Interface utilisateur et architecture finale 5

I Genèse et motivations

Les outils classiques de gestion de vulnérabilités (Dependabot, Snyk...) signalent une CVE dès qu'une bibliothèque est dans une plage de versions affectées. Ils ne tiennent pas compte du contexte réel d'utilisation dans le code source : la fonctionnalité vulnérable est-elle réellement appelée ? Les données contrôlées par l'attaquant atteignent-elles le sink vulnérable ? Des mitigations sont-elles en place ?

Cette approche génère un volume important de faux positifs. Pour une entreprise, cela se traduit concrètement par du temps perdu à trier des alertes non exploitables, une difficulté à prioriser les vraies menaces, et un risque de *alert fatigue* qui pousse les équipes à ignorer des vulnérabilités réellement critiques.

L'idée d'Argus est de répondre à cette question grâce à l'IA, tout en respectant une contrainte forte : en contexte entreprise, le code source ne doit pas quitter le serveur local. Toute l'architecture du projet découle de cette contrainte.

II Construction du graphe d'appels et premières frictions

La première brique fut la construction d'un graphe d'appels à partir du code source C++. tree-sitter a été retenu pour sa légèreté et son intégration Python, avec un parsing en deux passes : extraction des définitions de fonctions, puis extraction des appels dans chaque définition.

Problème : mauvaise attribution des fonctions à leurs bibliothèques

Lors des premiers tests avec le client, un problème fondamental est apparu : les CVEs remontées pour une fonction appartenaient parfois à la mauvaise bibliothèque. Par exemple, une fonction comme `send()` peut appartenir à la libc, à une bibliothèque réseau tierce, ou encore à une bibliothèque applicative — le parsing CMake seul ne suffit pas toujours à trancher.

Solution : ajout d'un **fallback LLM cloud** pour la résolution des symboles. Lorsque l'attribution automatique est ambiguë, un modèle cloud reçoit la liste des appels et des bibliothèques liées (CMake) — sans aucun code source — et retourne la bibliothèque la plus probable avec un score de confiance. Cela s'est révélé fiable sur les bibliothèques courantes (libcurl, OpenSSL, zlib, boost...).

Problème : versions affectées absentes de la NVD

En interrogeant l'API NVD pour récupérer les CVEs, on s'est rendu compte que certaines fiches ne listent pas explicitement les versions affectées. Sans cette information, impossible de savoir si la version utilisée dans le projet est réellement concernée.

Solution : ajout d'un second **fallback LLM cloud** : à partir de la description textuelle de la CVE, le modèle infère les plages de versions probablement affectées. Ici encore, aucun code source n'est transmis — seulement la description NVD de la vulnérabilité.

III Du code complet au BackSlicer

Première approche : analyse du code complet

L'idée initiale était simple : envoyer l'intégralité du code source au modèle on-premise pour qu'il juge de l'exploitabilité. En pratique, cette approche s'est heurtée à une limite immédiate : sur des projets réels, le code dépasse très largement la fenêtre de contexte des modèles on-premise (typiquement 8k à 32k tokens). Même avec de petits projets, l'ensemble des fichiers sources avec leurs includes représente bien plus que ce qu'un modèle embarqué peut traiter.

Le BackSlicer : extraire uniquement le contexte pertinent

L'idée du backward slicing est de n'envoyer au modèle que les lignes de code qui *influencent directement* les arguments de l'appel vulnérable, en remontant le flux de données depuis ce point. Le slicer fonctionne en 4 passes :

1. **Passe 1** : chaînes d'assignation (`a = b; b = recv(...)`)
2. **Passe 2** : sinks de sécurité (`memcpy`, `strcpy`, `malloc`...)
3. **Passe 3** : accès membres et indexation de tableaux sur les objets suivis
4. **Passe 4** : déclarations de taille (`sizeof`, `strlen`, constantes)

Le BackSlicer actuel est basé sur des heuristiques regex. Il est encore perfectible : les templates C++, les macros complexes et les appels via pointeurs de fonctions ne sont pas toujours correctement suivis. La solution idéale serait un slicer basé sur l'**AST Clang** (via `libclang`), qui produirait un vrai graphe de dépendances de données (PDG/SDG) avec une précision structurelle. Cette approche n'a cependant pas été retenue : elle impose de fournir l'ensemble des en-têtes système et des dépendances au moment de la compilation, une contrainte lourde à automatiser dans un outil générique. Le slicer heuristique constitue un compromis pragmatique — aucune configuration de l'environnement de compilation n'est requise, au prix d'une couverture incomplète sur le C++ avancé.

Rapports récursifs + rapport global

Le pipeline d'analyse a été structuré en deux niveaux. Le modèle on-premise génère d'abord un **rapport local** pour chaque nœud du graphe d'appels (chaque fonction potentiellement vulnérable), puis un **rapport global** qui compile l'ensemble des rapports locaux pour fournir une vue synthétique du risque projet.

IV Adaptation aux modèles on-premise

Le problème des faux négatifs

Le pipeline fonctionnait bien sur les grands modèles cloud (GPT-4, Claude...). Sur les modèles on-premise (24B paramètres), deux problèmes sont apparus :

- La fenêtre de contexte, même avec le BackSlicer, restait parfois insuffisante pour établir un verdict clair
- Quand le modèle n'était pas sûr, il avait tendance à assigner un score de 0.0 par prudence

En sécurité, un faux négatif (CVE exploitable non détectée) est bien plus grave qu'un faux positif (alerte sur un code en réalité sûr). Les prompts ont été refondus avec cette philosophie explicite : il vaut mieux sur-alerter que sous-alerter. Des plages de scores obligatoires ont été introduites par verdict (`INSUFFICIENT_INFO` ne peut jamais produire un score de 0.0).

Enrichissement CVE par LLM cloud

Pour compenser le manque de contexte du petit modèle, une étape d'enrichissement a été ajoutée avant l'analyse on-premise : un LLM cloud reçoit la description de la CVE et génère du contexte sémantique supplémentaire — exemples de mauvaise utilisation, pattern de code vulnérable canonique, prérequis d'exploitation, etc.

Ce contexte enrichi est injecté dans le prompt du modèle on-premise. L'**idée clé** : seule la description NVD est transmise au LLM cloud, jamais le code source — ce qui reste compatible avec la contrainte de confidentialité.

V Le workflow de benchmarking

Besoin d'évaluation objective

Pour prouver que la solution fonctionne et en mesurer les limites, il fallait un moyen d'évaluer objectivement la qualité des rapports produits par le modèle on-premise. L'idée adoptée : utiliser des **modèles juges cloud** (DeepSeek, Gemini...) qui analysent le même code et notent la qualité de l'analyse produite par le modèle on-premise.

Problème : divergence entre juges

Après mise en place du workflow, on s'est rendu compte que deux juges cloud ne donnaient pas systématiquement les mêmes scores sur un même rapport. La variance entre juges rendait la comparaison peu fiable.

Solution : mise en place d'un protocole de **consensus en 3 phases** : analyse indépendante, révision croisée avec justification technique obligatoire, puis synthèse par médiane pondérée. Le juge le plus proche de la médiane rédige le rapport final.

Contrainte de coût

Le benchmarking a un coût direct en tokens OpenRouter. Pour un projet modeste de 3–4 fichiers, un test complet (modèle testé + 2 juges × 3 phases) revient à environ **25 centimes**. Ce coût a limité le nombre de tests réalisables : seuls 12 tests ont été effectués sur 2 projets. Un échantillon plus large permettrait des conclusions statistiquement plus robustes.

VI Interface utilisateur et architecture finale

Design du frontend

Le frontend Next.js a été conçu avec un design sobre (palette navy foncé, accents bleus) autour du graphe d'appels interactif (vis-network). Les données transitent en deux appels distincts :

1. **Premier appel** (`/analyze`) : parsing du projet, construction du graphe d'appels, récupération des CVEs — retourne toutes les données structurées au frontend, sans appel IA
2. **Second appel** (`/llm_generate_report`) : génération des rapports IA sur les nœuds sélectionnés — seul ce second appel déclenche des requêtes vers le modèle on-premise

Rien n'est stocké côté serveur pour le workflow d'analyse (les données restent en mémoire le temps de la session). Seuls les résultats des tests de benchmarking sont persistés en base SQLite. Ce choix évite toute double analyse et garantit qu'aucune donnée de code ne s'accumule en dehors du contexte d'exécution.

Pistes d'amélioration des prompts

Un prompt d'analyse local représente actuellement 3 000 à 4 000 tokens, ce qui convient pour les modèles on-premise disposant d'une fenêtre de 32k tokens. Une piste d'amélioration serait la **décomposition en sous-prompts** : plutôt qu'un seul prompt dense, enchaîner plusieurs questions ciblées (version affectée ? flux de données ? guard contournable ?). Cette approche réduirait le risque d'hallucination et forcerait un raisonnement étape par étape, au prix d'une latence légèrement accrue.